# CPU Scheduling

Lecture Notes — Lecture 8
Concepts, Criteria, FCFS and SJF

Auqib Hamid Lone
*Department of Computer Science & Engineering, GCET Ganderbal*

October 29, 2026

Concepts That Click

## 8.1 Five Jobs, One CPU

Five students submit compile jobs to the department's lab server within five milliseconds of one another. The server has a single CPU. Table 8.1 lists, for each job, when it arrives and how long a CPU burst it needs. The question that organizes this chapter is the one the instructor faces at the console: *in what order should the jobs run, and how would we even decide that one order is better than another?*

| Process | P1 | P2 | P3 | P4 | P5 |
|---|---|---|---|---|---|
| Arrival time (ms) | 0 | 1 | 2 | 3 | 5 |
| CPU burst $\tau$ (ms) | 6 | 8 | 2 | 5 | 4 |

Table 8.1: The lab-server workload, the running example of this chapter.

We will schedule these five jobs three ways — first-come first-served (FCFS), shortest-job-first (SJF), and shortest-remaining-time-first (SRTF) — and measure each schedule with the same yardsticks. Four questions frame the chapter:

1. **Why schedule at all?** The CPU–I/O burst cycle, the scheduler, the dispatcher, and preemption (Section 8.2).
2. **What makes a schedule good?** Five criteria and the formulas for turnaround, waiting, and response time (Section 8.3).
3. **What does the simplest policy do?** FCFS and the convoy effect (Section 8.4).
4. **Can we do better?** SJF, its optimality, burst prediction, and the preemptive form SRTF (Sections 8.5–8.6).

Lecture 6 introduced the short-term scheduler as a box in the process-state diagram. This chapter opens that box and asks what policy it should run.

## 8.2 Basic Concepts

*Reading:* Silberschatz et al., Ch. 5 §5.1, pp. 183–187 [1]; Stallings, Ch. 9 §9.1, p. 406 [2].

### The CPU–I/O burst cycle

A process does not use the CPU continuously. It alternates between computing and waiting, and the waiting is what makes scheduling worthwhile: while one process waits for a disk or a keyboard, another can use the CPU.

**Definition (CPU burst and I/O burst).** *A CPU burst is an interval during which a process executes on the CPU; an I/O burst is an interval during which it waits for an I/O operation. Execution is a cycle of the two, beginning and ending with a CPU burst.*

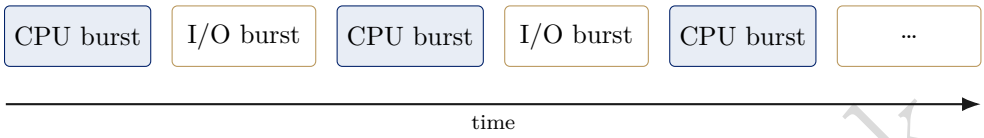



Figure 8.1: The CPU–I/O burst cycle: CPU bursts (blue) alternate with I/O bursts (gold outline) along the time axis — cf. Silberschatz et al., Ch. 5 §5.1, p. 183 [1].

Figure 8.1 reads left to right: a CPU burst, then an I/O burst, then another CPU burst, and so on until a final CPU burst ends in termination. Measured burst lengths are dominated by many short bursts and a few long ones. That distribution separates two kinds of program. An **I/O-bound** program has many short CPU bursts; a **CPU-bound** program has a few long ones. A good scheduler must serve both.

### The CPU scheduler and the ready queue

**Definition (CPU scheduler).** *The CPU scheduler (short-term scheduler) selects one process from the processes in memory that are ready to execute and allocates the CPU to it.*

The ready queue holds PCBs, not programs, and it is *not* necessarily a FIFO queue: it can be a FIFO queue, a priority queue, a tree, or an unordered list. The *policy* is the rule that picks from it, and FCFS, SJF, and SRTF are three such rules. Whenever one process waits for I/O, the scheduler gives the CPU to another — this is how multiprogramming keeps the CPU busy. The long-term and medium-term schedulers of Lecture 6 decide *who is in memory*; the CPU scheduler of this chapter decides *who runs*.

### Preemptive and non-preemptive scheduling

A scheduling decision may be needed at four kinds of moment, namely when a process:

1. switches from running to waiting (an I/O request, or `wait`);
2. switches from running to ready (a timer interrupt);
3. switches from waiting to ready (I/O completion);
4. terminates.

In cases 1 and 4 the running process gives up the CPU itself, so the scheduler has no choice but to pick another process. In cases 2 and 3 it has a choice.

**Definition (Non-preemptive and preemptive scheduling).** *Under non-preemptive (cooperative) scheduling, decisions are made only in cases 1 and 4: once a process has the CPU, it keeps*

Concepts That Click

*it until it blocks or exits. Under preemptive scheduling, decisions are also made in cases 2 and 3: the kernel may take the CPU away from a running process.*

Preemption has a price. If the kernel can stop a process at any moment, it may stop it in the middle of updating shared data, and the kernel must then protect that data from a half-finished update — the problem the concurrency unit of this course takes up.

**Example 8.1** (Which transitions force a decision)**.** *A process finishes its CPU burst and issues a disk* `read`*. Must a non-preemptive scheduler make a decision, and what about when the* `read` *later completes?*

Solution. *Issuing the* `read` *is case 1 (running → waiting): the CPU is now free, so the scheduler must pick some ready process. The completion of the* `read` *is case 3 (waiting → ready). A non-preemptive scheduler lets the currently running process keep the CPU; the newly ready process simply joins the ready queue. Only a preemptive scheduler may reconsider at case 3.*

### The dispatcher

**Definition (Dispatcher and dispatch latency).** *The dispatcher is the module that gives control of the CPU to the process the scheduler selected. The time it takes to stop one process and start another is the dispatch latency.*

The dispatcher does three things: it switches context (saving one PCB and loading another, as in Lecture 6), it switches to user mode, and it jumps to the saved program counter in the new process. It runs on *every* switch, so a policy that switches more often pays the dispatch latency more often. We will measure exactly that cost on the lab-server jobs in Section 8.7.

## 8.3 Scheduling Criteria

*Reading:* Silberschatz et al., Ch. 5 §5.2, pp. 187–188 [1].

Before comparing policies we need yardsticks: "better" means nothing until we say better for whom, measured how. Table 8.2 lists the five standard criteria.

| Criterion | Meaning | Goal |
|---|---|---|
| CPU utilization | fraction of time the CPU does useful work | maximize |
| Throughput | processes completed per unit time | maximize |
| Turnaround time (TAT) | submission to completion, per process | minimize |
| Waiting time (WT) | total time spent in the ready queue | minimize |
| Response time (RT) | submission to the *first* time on the CPU | minimize |

Table 8.2: The five scheduling criteria (Silberschatz et al., Ch. 5 §5.2).

Two observations shape everything that follows. First, the scheduling policy directly controls only the **waiting time**: it does not change how long a burst computes, only how long a process sits in the ready queue before and between its turns. Second, we usually optimize the *average* of a criterion, but interactive systems also care about the *variance* of response time: a predictable response is often worth more than a slightly faster but erratic one.

Concepts That Click

### The three per-process formulas

**Definition (TAT, WT, RT).** *For process* `Pi` *with arrival time* $\mathrm{AT}_i$*, CPU burst* $\tau_i$*, first-dispatch time* $\mathrm{ST}_i$*, and completion time* $\mathrm{CT}_i$ *(one CPU burst per process),*

$$\mathrm{TAT}_i = \mathrm{CT}_i - \mathrm{AT}_i, \qquad \mathrm{WT}_i = \mathrm{TAT}_i - \tau_i, \qquad \mathrm{RT}_i = \mathrm{ST}_i - \mathrm{AT}_i.$$

The waiting-time formula follows from the turnaround: between arrival and completion a process is either running (for exactly $\tau_i$ in total) or ready and not running, so the second part is $\mathrm{TAT}_i - \tau_i$. Hence WT counts *all* time ready but not running, including time spent back in the queue after a preemption. For a non-preemptive policy the process runs once, without interruption, starting at $\mathrm{ST}_i$; then $\mathrm{CT}_i = \mathrm{ST}_i + \tau_i$ and so $\mathrm{WT}_i = \mathrm{RT}_i$. Finally, idle CPU time is not anybody's waiting time: when the CPU is idle, no process is ready.

Gantt charts in this chapter draw time left to right; each bar is labelled with the process that runs, idle intervals are shaded grey, and arrivals are marked by gold arrows below the axis.

**Example 8.2** (Metrics from a schedule with an idle gap)**.** *Jobs* `P1` $(0, 3)$*,* `P2` $(5, 4)$*,* `P3` $(6, 2)$*, given as (AT,* $\tau$*) in ms, are scheduled FCFS. Find every metric and the CPU utilization.*

Solution. `P1` *arrives at 0 and runs from 0 to 3. At 3 nothing else has arrived, so the CPU idles from 3 to 5.* `P2` *arrives at 5 and starts at once, running 5 to 9.* `P3` *arrived at 6 and has been waiting; it runs 9 to 11. The chart is Figure 8.2.*
*Applying the formulas:* `P1`*:* $\mathrm{TAT}_1 = 3 - 0 = 3$*,* $\mathrm{WT}_1 = 3 - 3 = 0$*.* `P2`*:* $\mathrm{TAT}_2 = 9 - 5 = 4$*,* $\mathrm{WT}_2 = 4 - 4 = 0$ *(it starts at its arrival).* `P3`*:* $\mathrm{TAT}_3 = 11 - 6 = 5$*,* $\mathrm{WT}_3 = 5 - 2 = 3$*. Because FCFS is non-preemptive, each RT equals the WT.*

| *Proc.* | *AT* | $\tau$ | *CT* | *TAT* | *WT* | *RT* |
|---|---|---|---|---|---|---|
| `P1` | 0 | 3 | 3 | 3 | 0 | 0 |
| `P2` | 5 | 4 | 9 | 4 | 0 | 0 |
| `P3` | 6 | 2 | 11 | 5 | 3 | 3 |
| ***Average*** | | | | 4 | 1 | 1 |

*The CPU is busy for* $3 + 4 + 2 = 9$ *of the 11 ms, so utilization over* $[0, 11]$ *is* $9/11 \approx 81.8\%$*.*

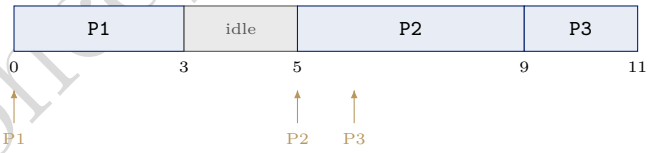



Figure 8.2: FCFS schedule of Example 8.2, with an idle gap from 3 to 5.

### Utilization and throughput include the dispatcher

**Example 8.3** (Dispatch latency on the lab server)**.** *The lab-server jobs have total burst* $6+8+2+5+4 = 25$ *ms. FCFS dispatches each job once, so there are 5 dispatches. With a dispatch latency of 0.5 ms per dispatch, the CPU spends* $25 + 5 \times 0.5 = 27.5$ *ms to finish all five jobs, of which 25 ms is useful work. Therefore*

$$\textit{utilization} = \frac{25}{27.5} \approx 90.9\%, \qquad \textit{throughput} = \frac{5\ \textit{jobs}}{27.5\ \textit{ms}} \approx 0.18\ \textit{jobs/ms}.$$

*With zero dispatch latency the same schedule gives* 100% *and* $5/25 = 0.2$ *jobs/ms. Every extra switch moves both numbers down.*

Concepts That Click

### Criteria conflict

Consider a laptop running a text editor, a browser, and a long video export. The user feels **response time**: the editor must react to a keystroke within milliseconds. A policy that maximized throughput might run the export to completion first and leave the editor frozen. Criteria conflict, and a scheduling policy is a choice of which one to favour. "Maximize throughput" is the wrong first goal for this machine.

## 8.4 First-Come, First-Served

*Reading:* Silberschatz et al., Ch. 5 §5.3.1, pp. 188–189 [1]; Stallings, Ch. 9 §9.2, p. 410 [2].

The simplest rule is the one at every ticket counter: serve whoever arrived first. It is fair-sounding and easy to build — and it can be very slow.

**Definition (FCFS).** *First-Come, First-Served (FCFS) allocates the CPU to the process that requested it first. The ready queue is a FIFO queue: an arriving PCB joins the tail; the scheduler takes the head.*

FCFS is **non-preemptive**: the running process keeps the CPU until it blocks or terminates. Its implementation is a linked list with a head and a tail pointer. Its average waiting time is often long, and it depends heavily on the order in which processes arrive, as the next two examples show.

**Example 8.4** (One long job first)**.** *`P1`, `P2`, `P3` arrive at time 0 in that order, with bursts 24, 3, and 3 ms. Under FCFS, `P1` runs 0–24, `P2` runs 24–27, and `P3` runs 27–30 (Figure 8.3). All arrive at 0, so each WT is simply the start time:* $\mathrm{WT} = 0, 24, 27$*. The average WT is* $(0+24+27)/3 = 17$ *ms. Two 3-ms jobs wait behind one 24-ms job.*




Figure 8.3: FCFS with the long job first (Example 8.4): average WT 17 ms.

**Example 8.5** (Same jobs, different arrival order)**.** *Now `P2`, `P3`, `P1` arrive at time 0 in that order. FCFS runs `P2` 0–3, `P3` 3–6, `P1` 6–30 (Figure 8.4). The waiting times are* $\mathrm{WT}_1 = 6$, $\mathrm{WT}_2 = 0$, $\mathrm{WT}_3 = 3$*, and the average is* $(6+0+3)/3 = 3$ *ms — a factor of* $17/3 \approx 5.7$ *better than Example 8.4, with the same work and the same finish time of 30 ms.*




Figure 8.4: FCFS with the short jobs first (Example 8.5): average WT 3 ms.

### The convoy effect

Example 8.4 is the static picture of a dynamic problem. Suppose one **CPU-bound** process holds the CPU for a long burst. Meanwhile many **I/O-bound** processes finish their I/O and queue behind it, and the I/O devices sit idle. Eventually the CPU-bound process blocks for I/O; the short jobs run quickly and also block on I/O; now the CPU sits idle. The cycle repeats, and

Concepts That Click

everyone moves at the speed of the slowest process. This is the **convoy effect**. Its consequence is that FCFS gives lower CPU and device utilization than letting the short jobs go first; and because it is non-preemptive, FCFS is unsuitable for time-sharing systems, where every user needs a share of the CPU at regular intervals (Silberschatz et al., Ch. 5 §5.3.1, pp. 188–189).

**Example 8.6** (The lab server under FCFS)**.** *Order of arrival is* `P1`*,* `P2`*,* `P3`*,* `P4`*,* `P5`*, so FCFS runs them in that order back to back:* `P1` *0–6,* `P2` *6–14,* `P3` *14–16,* `P4` *16–21,* `P5` *21–25 (Figure 8.5). For example,* `P3` *arrives at 2 and finishes at 16, so* $\mathrm{TAT}_3 = 14$ *and* $\mathrm{WT}_3 = 14 - 2 = 12$*: it needs only 2 ms but waits 12 ms behind* `P2`*'s 8-ms burst. The full table is:*

| *Proc.* | *AT* | $\tau$ | *CT* | *TAT* | *WT* | *RT* |
|---|---|---|---|---|---|---|
| `P1` | 0 | 6 | 6 | 6 | 0 | 0 |
| `P2` | 1 | 8 | 14 | 13 | 5 | 5 |
| `P3` | 2 | 2 | 16 | 14 | 12 | 12 |
| `P4` | 3 | 5 | 21 | 18 | 13 | 13 |
| `P5` | 5 | 4 | 25 | 20 | 16 | 16 |
| ***Average*** | | | | *14.2* | *9.2* | *9.2* |

*The waiting times sum to* $0 + 5 + 12 + 13 + 16 = 46$*, so the average WT is* $46/5 = 9.2$ *ms.*

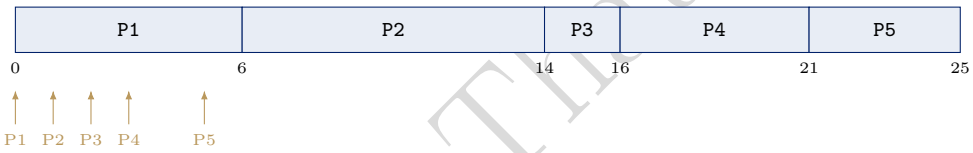



Figure 8.5: The lab-server jobs under FCFS (Example 8.6).

## 8.5 Shortest-Job-First

*Reading:* Silberschatz et al., Ch. 5 §5.3.2, pp. 189–192 [1]; Stallings, Ch. 9 §9.2 [2].

If order matters so much, pick the order deliberately: run the shortest job first.

**Definition (SJF).** *Shortest-Job-First (SJF) gives the CPU to the ready process with the smallest next CPU burst. Ties are broken FCFS.*

The more accurate name is *shortest-next-CPU-burst*: the length that matters is the next burst, not the whole job. In the form we study first, SJF is **non-preemptive**: once chosen, a process runs its whole burst. Its preemptive form is SRTF (Section 8.6). Stallings calls the non-preemptive policy SPN, *shortest process next.*

**Example 8.7** (SJF with simultaneous arrivals)**.** `P1`*–*`P4` *arrive at 0 with bursts 6, 8, 7, 3 ms. SJF sorts by burst:* `P4` *(3) runs 0–3,* `P1` *(6) runs 3–9,* `P3` *(7) runs 9–16,* `P2` *(8) runs 16–24 (Figure 8.6). The waiting times are the start times:* $\mathrm{WT}_1 = 3$, $\mathrm{WT}_2 = 16$, $\mathrm{WT}_3 = 9$, $\mathrm{WT}_4 = 0$*, average* $(3 + 16 + 9 + 0)/4 = 7$ *ms. FCFS in index order would run* `P1`*,* `P2`*,* `P3`*,* `P4` *starting at 0, 6, 14, 21, for an average of* $(0 + 6 + 14 + 21)/4 = 10.25$ *ms.*

Concepts That Click

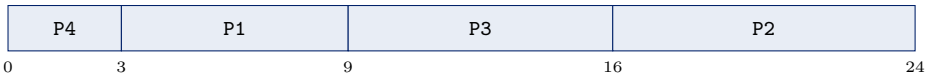



Figure 8.6: SJF on four simultaneous arrivals (Example 8.7): average WT 7 ms, versus 10.25 ms for FCFS.

### Why SJF is optimal

Let $n$ processes all be ready at time 0 and run them in some order with bursts $\tau_{(1)}, \tau_{(2)}, \ldots, \tau_{(n)}$. The process in position $k$ waits for the bursts of all processes before it, so the $k$-th burst delays every one of the $n-k$ processes after it. Summing over processes,

$$\sum_i \mathrm{WT}_i = \sum_{k=1}^{n} (n-k)\,\tau_{(k)}.$$

The weights $n-1, n-2, \ldots, 0$ decrease with position, and the largest weight $n-1$ multiplies the first burst. To minimize the sum, the smallest burst must take the largest weight, the next smallest the next weight, and so on: that is exactly SJF order. Equivalently, if two adjacent processes appear long–short, swapping them into short–long order lowers the total: with the long burst $L$ in position $k$ and the short burst $S$ in position $k+1$, the swap changes the sum by $(n-k)S + (n-k-1)L - (n-k)L - (n-k-1)S = S - L < 0$.

**Definition (Optimality of SJF).** *For a given set of processes, SJF gives the minimum average waiting time.*

(Silberschatz et al., Ch. 5 §5.3.2, from p. 189, states that SJF is provably optimal.)

**Example 8.8** (The lab server under non-preemptive SJF)**.** *At $t = 0$ only `P1` is ready, so SJF has no choice: `P1` runs to completion at 6. By then `P2`, `P3`, `P4`, `P5` have all arrived, with bursts $\{8, 2, 5, 4\}$. SJF runs them shortest first: `P3` 6–8, `P5` 8–12, `P4` 12–17, `P2` 17–25 (Figure 8.7).*

| *Proc.* | *AT* | $\tau$ | *CT* | *TAT* | *WT* | *RT* |
|---|---|---|---|---|---|---|
| *P1* | *0* | *6* | *6* | *6* | *0* | *0* |
| *P2* | *1* | *8* | *25* | *24* | *16* | *16* |
| *P3* | *2* | *2* | *8* | *6* | *4* | *4* |
| *P4* | *3* | *5* | *17* | *14* | *9* | *9* |
| *P5* | *5* | *4* | *12* | *7* | *3* | *3* |
| ***Average*** | | | | *11.4* | *6.4* | *6.4* |

*The waiting times sum to $0 + 16 + 4 + 9 + 3 = 32$, so the average WT is $32/5 = 6.4$ ms, against 9.2 ms under FCFS.*

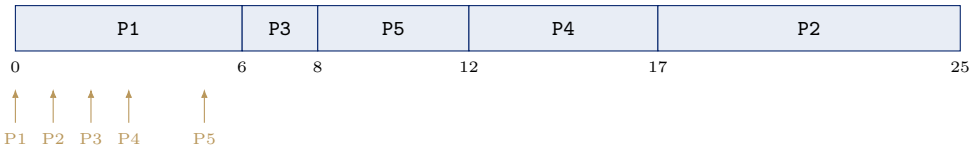



Figure 8.7: The lab-server jobs under non-preemptive SJF (Example 8.8).

Concepts That Click

### The catch: predicting the next burst

The short-term scheduler cannot read the future. It knows how long past bursts took, but not how long the next one will take, so it can only *predict* the next CPU burst. The standard predictor is the exponential average.

**Definition (Exponential averaging).** *Let $\tau_n$ be the measured length of the $n$-th CPU burst and $\hat{\tau}_n$ its prediction. The prediction for the next burst is*

$$\hat{\tau}_{n+1} = \alpha\,\tau_n + (1-\alpha)\,\hat{\tau}_n, \qquad 0 \le \alpha \le 1.$$

The parameter $\alpha$ sets how much recent behaviour counts. With $\alpha = 0$, $\hat{\tau}_{n+1} = \hat{\tau}_n$: only history counts, and recent behaviour is ignored. With $\alpha = 1$, $\hat{\tau}_{n+1} = \tau_n$: only the last burst counts. Expanding the recursion,

$$\hat{\tau}_{n+1} = \alpha\tau_n + (1-\alpha)\alpha\tau_{n-1} + \cdots + (1-\alpha)^j\alpha\tau_{n-j} + \cdots + (1-\alpha)^{n+1}\hat{\tau}_0,$$

so the burst $j$ steps back gets weight $\alpha(1-\alpha)^j$: each step back counts less. (Silberschatz writes $t_n$ for the measured burst; these notes keep $\tau$ for bursts and add a hat for a prediction.)

**Example 8.9** (Predicting bursts with $\alpha = 1/2$)**.** *Take $\hat{\tau}_0 = 10$ ms and measured bursts $6, 4, 6, 4, 13, 13, 13$ ms. Each prediction is the average of the last measurement and the last prediction:*

$$\begin{aligned}
\hat{\tau}_1 &= \tfrac{1}{2}(6) + \tfrac{1}{2}(10) = 8, & \hat{\tau}_2 &= \tfrac{1}{2}(4) + \tfrac{1}{2}(8) = 6,\\
\hat{\tau}_3 &= \tfrac{1}{2}(6) + \tfrac{1}{2}(6) = 6, & \hat{\tau}_4 &= \tfrac{1}{2}(4) + \tfrac{1}{2}(6) = 5,\\
\hat{\tau}_5 &= \tfrac{1}{2}(13) + \tfrac{1}{2}(5) = 9, & \hat{\tau}_6 &= \tfrac{1}{2}(13) + \tfrac{1}{2}(9) = 11,\\
\hat{\tau}_7 &= \tfrac{1}{2}(13) + \tfrac{1}{2}(11) = 12.
\end{aligned}$$

| $n$ | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| measured $\tau_n$ | 6 | 4 | 6 | 4 | 13 | 13 | 13 | — |
| predicted $\hat{\tau}_n$ | 10 | 8 | 6 | 6 | 5 | 9 | 11 | 12 |

*When the behaviour jumps from 4 to 13, the prediction lags — 9, 11, 12 — and converges toward 13. This is the same instance as the exponential-average illustration in Silberschatz et al., Ch. 5 §5.3.2.*

**Example 8.10** (Choosing $\alpha$ for an alternating process)**.** *A process alternates bursts of 2 ms and 20 ms. With $\alpha$ near 1 the predictor follows the last burst: it predicts 2 right before a 20 and 20 right before a 2 — always wrong. With $\alpha$ near 0 the prediction settles near the long-run average, about 11 ms: never exact, but stable. SJF is only as good as its predictor; a bad guess turns "shortest first" into an arbitrary order.*

## 8.6 Shortest-Remaining-Time-First

*Reading:* Silberschatz et al., Ch. 5 §5.3.2 (from p. 189) [1]; Stallings, Ch. 9 §9.2 [2].

Non-preemptive SJF ignores a short job that arrives one millisecond after a long one starts. The preemptive form does not.

Concepts That Click

**Definition (SRTF).** *Shortest-Remaining-Time-First (SRTF) is preemptive SJF. When a process arrives, its burst is compared with the remaining time of the running process; if it is strictly shorter, the running process is preempted.*

Decisions now happen at every arrival and every completion (cases 1, 3, and 4 of Section 8.2). Equal remaining times do not preempt, and ties go to the earlier arrival. Waiting time now includes the time a preempted process spends back in the ready queue, which is why we need the general formula $\mathrm{WT}_i = \mathrm{TAT}_i - \tau_i$ rather than "start time minus arrival". Stallings calls this policy SRT.

**Example 8.11** (SRTF versus non-preemptive SJF)**.** *Processes (AT, $\tau$): P1 $(0, 8)$, P2 $(1, 4)$, P3 $(2, 9)$, P4 $(3, 5)$. Step by step:*

- $t = 0$*: only P1; it starts.*
- $t = 1$*: P2 arrives with 4; P1 has $8 - 1 = 7$ left. $4 < 7$, so P1 is preempted and P2 runs.*
- $t = 2$*: P3 arrives with 9; P2 has 3 left. No preemption.*
- $t = 3$*: P4 arrives with 5; P2 has 2 left. No preemption.*
- $t = 5$*: P2 completes. Remaining: P1 7, P3 9, P4 5. P4 runs 5–10.*
- $t = 10$*: P1 (7) beats P3 (9); P1 resumes and runs 10–17. Then P3 runs 17–26.*

| *Proc.* | *AT* | $\tau$ | *CT* | *TAT* | *WT* | *RT* |
|---|---|---|---|---|---|---|
| *P1* | *0* | *8* | *17* | *17* | *9* | *0* |
| *P2* | *1* | *4* | *5* | *4* | *0* | *0* |
| *P3* | *2* | *9* | *26* | *24* | *15* | *15* |
| *P4* | *3* | *5* | *10* | *7* | *2* | *2* |
| ***Average*** | | | | *13* | *6.5* | *4.25* |

*P1 waited from 1 to 10, so* $\mathrm{WT}_1 = 9$ *even though its* $\mathrm{RT}_1 = 0$*. The average WT is* $(9+0+15+2)/4 = 26/4 = 6.5$ *ms. Run non-preemptively, SJF would execute P1 0–8, P2 8–12, P4 12–17, P3 17–26, with waits* $0, 7, 15, 9$ *and average* $31/4 = 7.75$ *ms.*

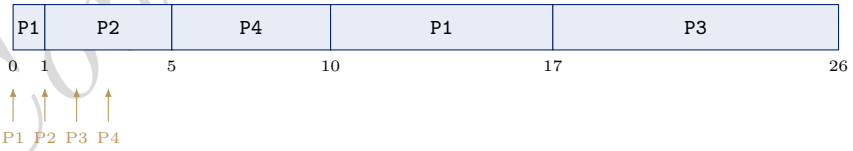



Figure 8.8: SRTF schedule of Example 8.11: P1 is preempted at 1 and resumes at 10.

**Example 8.12** (The lab server under SRTF)**.** *P1 starts at 0. At $t = 1$, P2 (8) does not beat P1's remaining 5. At $t = 2$, P3 (2) beats P1's remaining 4: P1 is preempted and P3 runs 2–4. At $t = 3$, P4 (5) does not beat P3's remaining 1. At $t = 4$, P3 completes; the remaining times are P1 4, P2 8, P4 5, so P1 resumes. At $t = 5$, P5 (4) does not beat P1's remaining 3, so P1 finishes at 8. Then P5 (4) runs 8–12, P4 (5) 12–17, and P2 (8) 17–25 (Figure 8.9).*

Concepts That Click

| *Proc.* | *AT* | $\tau$ | *CT* | *TAT* | *WT* | *RT* |
|---|---|---|---|---|---|---|
| *P1* | *0* | *6* | *8* | *8* | *2* | *0* |
| *P2* | *1* | *8* | *25* | *24* | *16* | *16* |
| *P3* | *2* | *2* | *4* | *2* | *0* | *0* |
| *P4* | *3* | *5* | *17* | *14* | *9* | *9* |
| *P5* | *5* | *4* | *12* | *7* | *3* | *3* |
| ***Average*** | | | | *11* | *6* | *5.6* |

*The waiting times sum to* $2+16+0+9+3=30$*: average WT* $30/5=6.0$ *ms. The response times sum to 28: average RT 5.6 ms.*

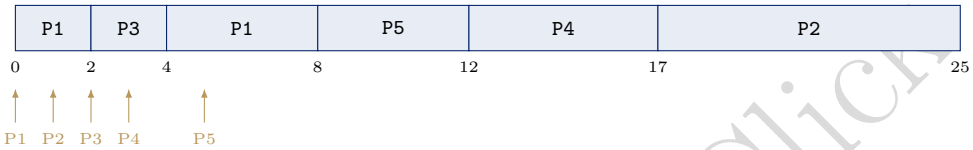



Figure 8.9: The lab-server jobs under SRTF (Example 8.12).

## 8.7 Comparing the Three Policies

Table 8.3 collects the averages of Examples 8.6, 8.8, and 8.12.

| Policy | avg WT | avg TAT | avg RT | dispatches |
|---|---|---|---|---|
| FCFS | 9.2 | 14.2 | 9.2 | 5 |
| SJF (non-preemptive) | 6.4 | 11.4 | 6.4 | 5 |
| SRTF | 6.0 | 11.0 | 5.6 | 6 |

Table 8.3: Three policies on the lab-server workload (all times in ms).

All three schedules finish at 25 ms: the work is the same, and only the *order* — and hence the waiting — changes. SRTF buys the best averages with one extra dispatch (its Gantt chart has six bars). With the 0.5-ms dispatch latency of Example 8.3, SRTF's utilization is $25/(25+6\times0.5)=25/28\approx89.3\%$, against 90.9% for FCFS.

### The price of short-first: starvation

If short jobs keep arriving, a long job can wait indefinitely: SJF and SRTF can **starve** long processes. On the lab server, `P2` (8 ms) already has the largest WT (16 ms) under both SJF and SRTF; a steady stream of 2-ms jobs would postpone it forever. FCFS never starves anyone, because each process waits only for those ahead of it in the queue.

SJF and SRTF carry two further costs: they need a burst prediction, and SRTF pays a dispatch on every preemption. Ageing — raising a process's priority with the time it has waited — is the usual cure for starvation, and is taken up in Lecture 9.

Concepts That Click

### Common misconceptions

- "Waiting time is completion minus arrival." That is TAT; WT = TAT − $\tau$.
- "SJF uses the total length of the job." It uses the predicted *next CPU burst*.
- "SRTF preempts on equal remaining time." Only a strictly shorter newcomer preempts.
- "A better policy finishes all the work sooner." The last completion is the same; the averages change.

## 8.8 Summary

- Processes alternate **CPU bursts** and **I/O bursts**; the short-term scheduler fills the CPU while others wait for I/O.
- Scheduling happens on block and exit (non-preemptive) and also on timer or I/O completion (**preemptive**); the **dispatcher** pays a latency on every switch.
- TAT = CT − AT, WT = TAT − $\tau$, RT = ST − AT.
- **FCFS** is simple and starvation-free but suffers the convoy effect.
- **SJF** minimizes average WT but needs a predicted burst (exponential average); **SRTF** is its preemptive form. Both can starve long jobs.

## 8.9 Looking Ahead and Further Reading

This chapter covered what scheduling is, how to measure it, and the two policies that order by arrival and by burst length. The next lecture covers **Priority** scheduling and ageing, **Round Robin** with a time quantum, multilevel queues and feedback queues, and scheduling on more than one CPU. Reading for it: Silberschatz et al., Ch. 5 §5.3–5.5 [1]; Stallings, Ch. 9 §9.2 and Ch. 10 §10.1 [2].

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 5 §5.1 (basic concepts, p. 183), §5.2 (scheduling criteria, p. 187), §5.3.1 (FCFS, p. 188), §5.3.2 (SJF and SRTF, p. 189).
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 9 §9.1 (types of scheduling, p. 406), §9.2 (scheduling algorithms: FCFS, SPN, SRT, p. 410).

## Self-Check

### Exercises

1. Processes (AT, $\tau$) in ms: `P1` $(0, 3)$, `P2` $(1, 5)$, `P3` $(2, 1)$, `P4` $(3, 2)$. Draw the FCFS and the non-preemptive SJF Gantt charts and give the average WT of each.
2. Processes (AT, $\tau$): `P1` $(0, 5)$, `P2` $(1, 1)$, `P3` $(2, 3)$. Compute the average WT under non-preemptive SJF and under SRTF, and the number of dispatches SRTF makes.

Concepts That Click

3. With $\alpha = 0.25$ and $\hat{\tau}_0 = 8$ ms, the first three measured bursts are 4, 12, 6 ms. Compute $\hat{\tau}_1$, $\hat{\tau}_2$, $\hat{\tau}_3$.

4. Three processes arrive at 0 with bursts 5, 2, 1 ms. Using the formula $\sum_i \mathrm{WT}_i = \sum_k (n-k)\tau_{(k)}$, compute the average WT for the order 5, 2, 1 and for SJF order, and explain why no order can do better than SJF.

5. Explain why, for a non-preemptive policy, $\mathrm{WT}_i = \mathrm{RT}_i$, and give a one-line counterexample under SRTF from Example 8.11.

### Solutions

1. *FCFS:* `P1` 0–3, `P2` 3–8, `P3` 8–9, `P4` 9–11. WT $= 0, 2, 6, 6$; average $14/4 = 3.5$ ms. *SJF:* at 0 only `P1`, which runs 0–3. At 3 the ready bursts are `P2` 5, `P3` 1, `P4` 2: `P3` 3–4, `P4` 4–6, `P2` 6–11. WT: `P1` 0, `P2` $6 - 1 = 5$, `P3` $3 - 2 = 1$, `P4` $4 - 3 = 1$; average $7/4 = 1.75$ ms.

2. *SJF:* `P1` runs 0–5 (only process at 0); at 5, `P2` (1) then `P3` (3): `P2` 5–6, `P3` 6–9. WT $= 0, 4, 4$; average $8/3 \approx 2.67$ ms. *SRTF:* at 1, `P2` (1) beats `P1`'s remaining 4 and runs 1–2; at 2, `P3` (3) beats `P1`'s remaining 4 and runs 2–5; `P1` resumes 5–9. CT $= 9, 2, 5$; TAT $= 9, 1, 3$; WT $= 4, 0, 0$; average $4/3 \approx 1.33$ ms. SRTF makes 4 dispatches (`P1`, `P2`, `P3`, `P1`).

3. $\hat{\tau}_1 = 0.25(4) + 0.75(8) = 7$; $\hat{\tau}_2 = 0.25(12) + 0.75(7) = 8.25$; $\hat{\tau}_3 = 0.25(6) + 0.75(8.25) = 7.6875$ ms.

4. Order 5, 2, 1: $\sum \mathrm{WT} = 2(5) + 1(2) + 0(1) = 12$, average 4 ms. SJF order 1, 2, 5: $2(1) + 1(2) + 0(5) = 4$, average $4/3 \approx 1.33$ ms. The weights $2, 1, 0$ decrease with position, so the sum is smallest when the smallest burst takes the largest weight; any long–short adjacent pair can be swapped to lower the sum by $L - S > 0$, so only the sorted order cannot be improved.

5. Non-preemptively, process $i$ runs once from $\mathrm{ST}_i$ to $\mathrm{CT}_i = \mathrm{ST}_i + \tau_i$, so $\mathrm{WT}_i = \mathrm{CT}_i - \mathrm{AT}_i - \tau_i = \mathrm{ST}_i - \mathrm{AT}_i = \mathrm{RT}_i$. Under SRTF in Example 8.11, `P1` has $\mathrm{RT}_1 = 0$ but $\mathrm{WT}_1 = 9$, because it waited from 1 to 10 after preemption.

## References


[1] Abraham Silberschatz, Peter B. Galvin, and Greg Gagne. *Operating System Concepts*. John Wiley and Sons, 2018.

[2] William Stallings. *Operating Systems: Internals and Design Principles*. Prentice Hall, 2001.


Concepts That Click